

PROJECT TITLE:

ROAD TRAFFIC ANALYSIS SYSTEM

SUBJECT: Data Structure and Algorithm

LAB INCHARGE: Maryam

Group Members:-

Names	IDs	Topics covered
Waqas	**240756**	**Graphs, BST**
Radia	**249529**	**Linklists,Queues**
S aqlain	**243370**	**Basics Algorithm**

Contents

1. ABSTRACT 3
2. OBJECTIVES OF THE PROJECT 3
3. DATA STRUCTURES USED 4
   3.1 Graph 4
   3.2 Linked List 4
   3.3 Binary Search Tree (BST) 5
   3.4 Queue 5
   3.5 Arrays / Vectors 5
4. ALGORITHMS USED 6
   4.1 Breadth First Search (BFS) 6
   4.2 Depth First Search (DFS) 6
   4.3 Dijkstra's Algorithm 6
5. CLASS-WISE DESCRIPTION 7
   Intersection Class 7
   TrafficNode & TrafficHistory Classes 7
   RoadSearchBST Class 7
   Graph Class 8
   MenuUI Class 8
6. METHODOLOGY 8
7. FEATURES 8
8. LIMITATIONS 9
9. FUTURE IMPROVEMENTS 9
11. CODE: 10
12. OUTPUT 26
13. Conclusion 27

1. ABSTRACT

Traffic congestion is a major problem in modern cities, causing delays, fuel wastage, and increased pollution. This project, Road Traffic Analysis System, is designed to simulate and analyze city traffic using fundamental Data Structures and Algorithms (DSA).

The system models intersections as nodes and roads as edges using graph data structures. It provides functionality such as shortest path calculation using Dijkstra's algorithm, traffic congestion detection, road searching using Binary Search Tree (BST), and traffic history management using Linked Lists.

The project is implemented in C++, following object-oriented programming principles. This system helps in understanding how DSA concepts are applied in real-world traffic management systems.

2. OBJECTIVES OF THE PROJECT

- The main objectives of this project are:
- To represent a city road network using **graphs**
- To find the **shortest route** between two intersections
- To analyze **traffic congestion**
- To store and manage **traffic history**
- To provide fast **road searching**
- To apply **multiple DSA concepts in a single project**
- To improve understanding of **real-world applications of DSA**

3. DATA STRUCTURES USED

This project uses different data structures to solve traffic-related problems in an organized way. Each data structure is selected based on its importance and suitability for a specific task in the system. The use of these structures makes the system efficient, easy to understand, and closer to real-world traffic analysis.

By combining multiple data structures, the project demonstrates how different concepts of DSA work together in a single application. This also helps in improving problem-solving and logical thinking skills.

3.1 Graph

Graph is the **main data structure** used in this project because a city road network naturally forms a graph. In this system, **intersections are represented as vertices**, and **roads are represented as edges**. This makes it easy to model real-world traffic flow.

Two graph representations are used. The **adjacency list** is used for BFS and DFS traversal because it is efficient for visiting connected nodes. The **adjacency matrix** is used to store distance and traffic level, which helps in shortest path calculation and congestion detection.

3.2 Linked List

Linked List is used to store **traffic history records** in the system. Each record is stored as a node containing location, vehicle count, time, and a pointer to the next record. Records are added dynamically as traffic data changes.

Linked List is suitable here because traffic records are frequently updated. It allows easy insertion of new records and deletion of old records without wasting memory, which makes the system flexible and efficient.

3.3 Binary Search Tree (BST)

Binary Search Tree is used for road searching in this project. Roads are stored using names such as `0-1`, which represent the connection between two intersections. This helps in organizing road data in a sorted manner.

BST makes searching faster compared to linear search. It also allows roads to be displayed in sorted order using inorder traversal, which improves data organization and readability.

3.4 Queue

Queue is used in the Breadth First Search (BFS) algorithm. BFS requires visiting nodes level by level, and this process follows the First In First Out (FIFO) principle.

Queue ensures that the intersection added first is processed first. This makes traversal systematic and helps in exploring nearby intersections before moving to farther ones.

3.5 Arrays / Vectors

Arrays and vectors are used to store different types of data in the project. Vectors are mainly used to store adjacency lists, distances, and visited status of nodes.

Vectors are preferred because they provide dynamic size and easy access to elements. They also help in simplifying the implementation of graph algorithms.

4. ALGORITHMS USED

Different algorithms are used in this project to analyze traffic efficiently. These algorithms help in traversal, shortest path calculation, and understanding connectivity between intersections.

The use of standard algorithms improves the performance of the system and provides correct and reliable results for traffic analysis.

4.1 Breadth First Search (BFS)

BFS is used to traverse the graph level by level. It starts from a given intersection and explores all nearby intersections before moving to the next level.

BFS uses a queue to maintain the order of traversal.
Time Complexity: O(V + E), where V is vertices and E is edges.

4.2 Depth First Search (DFS)

DFS explores the graph by going **deep into one path** before backtracking. It is implemented using recursion in this project.

DFS helps in understanding connectivity and structure of the graph.
Time Complexity: O(V + E).

4.3 Dijkstra's Algorithm

Dijkstra's Algorithm is used to find the **shortest distance** between two intersections. It calculates minimum distance using a distance array and visited array.

This helps users choose the shortest route in the city.
Time Complexity: $O(V^2)$.

5. CLASS-WISE DESCRIPTION

The project is divided into different classes to follow **object-oriented programming** principles. Each class has a specific role, which makes the code modular and easy to manage.

This approach also improves readability and allows future expansion of the system.

Intersection Class

The Intersection class stores basic information about an intersection. It contains an ID and name for each intersection.

This class helps in uniquely identifying and displaying intersections in the city map.

TrafficNode & TrafficHistory Classes

These classes manage traffic history using a linked list. TrafficNode represents a single record, while TrafficHistory manages all records.

Functions include inserting records, displaying traffic history, and deleting the oldest record when required.

RoadSearchBST Class

This class manages road searching using a Binary Search Tree. Roads are inserted and searched efficiently.

It also provides sorted display of roads using inorder traversal.

Graph Class

Graph class is the **core class** of the system. It manages intersections, roads, and traffic analysis.

It includes functions for BFS, DFS, shortest path calculation, and traffic congestion detection.

MenuUI Class

MenuUI class provides a **menu-based interface** for the user. It connects all system components.

It allows users to interact with the system easily without knowing internal implementation details.

6. METHODOLOGY

First, intersections are created in the system. Then, roads are added between intersections with distance and traffic level information.

Traffic data is stored, and algorithms are applied to analyze routes and congestion. Results are displayed using a menu-driven system.

7. FEATURES

The system provides a simple and user-friendly menu. Users can calculate shortest paths and detect traffic congestion easily.

It also supports traffic history management and fast road searching using BST.

8. LIMITATIONS

The system supports a fixed number of intersections. It does not use real-time traffic data.

The project is console-based and uses a basic version of Dijkstra's algorithm.

9. FUTURE IMPROVEMENTS

Dijkstra's algorithm can be optimized using a priority queue. File handling can be added for permanent data storage.

The city size can be increased, and a graphical interface can be developed.

11. CODE:

Libraries

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <limits.h>
using namespace std;
```

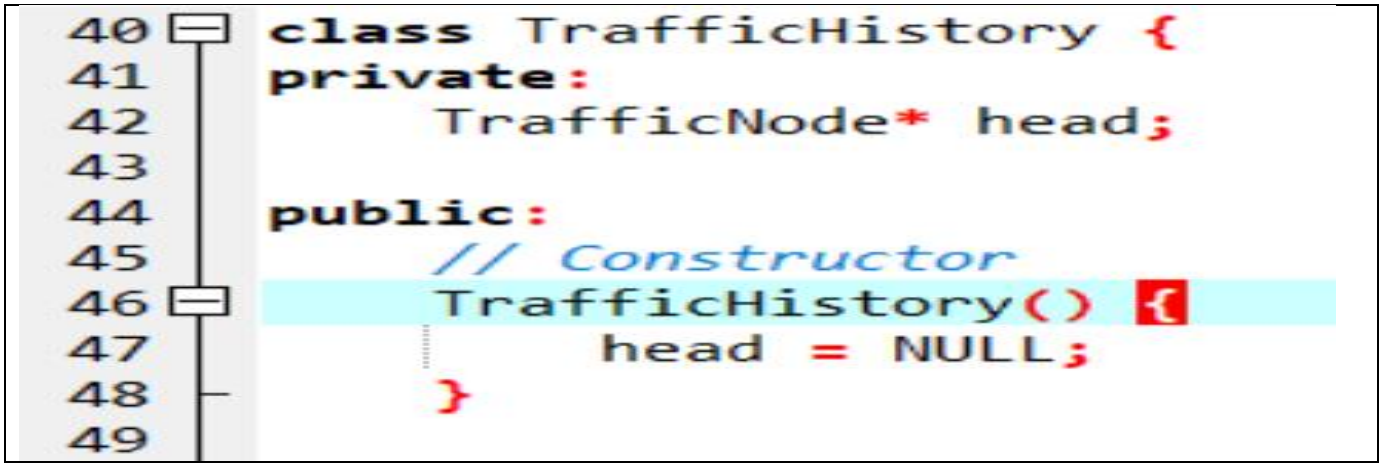
CLASS INTERSECTION

```cpp
// ========== INTERSECTION CLASS ==========
// Represents a location/node in the city
class Intersection {
public:
    int id;
    string name;
    // Constructor to initialize intersection
    Intersection(int i, string n) {
        id = i;
        name = n;}
    // Display intersection information
    void displayInfo() {
        cout << "ID: " << id << " | Name: " << name << endl;}};
// ========== TRAFFIC RECORD NODE (LINKED LIST) ==========
// Used to store daily traffic history
```

CLASS TRAFFICNODE

```
22  class TrafficNode {
23  public:
24      string location;
25      int vehicleCount;
26      string timeStamp;
27      TrafficNode* next;
28
29      // Constructor
30      TrafficNode(string loc, int count, string time) {
31          location = loc;
32          vehicleCount = count;
33          timeStamp = time;
34          next = NULL;
35      }
36  };
```

class TrafficHistory


40 class TrafficHistory {
41 private:
42 TrafficNode* head;
43
44 public:
45 // Constructor
46 TrafficHistory() {
47 head = NULL;
48 }
49

```
49
50      // Insert new traffic record at beginning
51      void insertRecord(string loc, int count, string time) {
52          TrafficNode* newNode = new TrafficNode(loc, count, time);
53          newNode->next = head;
54          head = newNode;
55          cout << "Traffic record added successfully!" << endl;
56      }
57
58      // Display all traffic records
59      void displayRecords() {
60          if (head == NULL) {
61              cout << "No traffic records found!" << endl;
62              return;
63          }
64
65          cout << "\n===== TRAFFIC HISTORY =====" << endl;
66          TrafficNode* temp = head;
67          int count = 1;
68          while (temp != NULL) {
69              cout << count << ". Location: " << temp->location
70                   << " | Vehicles: " << temp->vehicleCount
71                   << " | Time: " << temp->timeStamp << endl;
72              temp = temp->next;
73              count++;
74          }
```

```
75      // Delete oldest record (last node)
76      void deleteOldRecord() {
77          if (head == NULL) {
78              cout << "No records to delete!" << endl;
79              return;
80          }
81
82          if (head->next == NULL) {
83              delete head;
84              head = NULL;
85              cout << "Last record deleted!" << endl;
86              return;
87          }
88
89          TrafficNode* temp = head;
90          while (temp->next->next != NULL) {
91              temp = temp->next;
92          }
93          delete temp->next;
94          temp->next = NULL;
95          cout << "Oldest record deleted!" << endl;
96      }
97  };
```

CLASS BSTNODE

```
 97  class BSTNode {
 98  public:
 99      string roadName;
100      int distance;
101      BSTNode* left;
102      BSTNode* right;
103
104      // Constructor
105      BSTNode(string name, int dist) {
106          roadName = name;
107          distance = dist;
108          left = NULL;
109          right = NULL;
110      }
111  };
```

CLASS BST SEARCH

```
115  class RoadSearchBST {
116  private:
117      BSTNode* root;
118
119      // Helper function to insert road in BST
120      BSTNode* insertHelper(BSTNode* node, string name, int dist) {
121          if (node == NULL) {
122              return new BSTNode(name, dist);
123          }
124
125          if (name < node->roadName) {
126              node->left = insertHelper(node->left, name, dist);
127          } else {
128              node->right = insertHelper(node->right, name, dist);
129          }
130          return node;
```

```
133     // Helper function for inorder display
134     void inorderHelper(BSTNode* node) {
135         if (node == NULL) return;
136 
137         inorderHelper(node->left);
138         cout << "Road: " << node->roadName <<
139         " | Distance: " << node->distance << " km" << end
140         inorderHelper(node->right);
141     }
142 
143     // Helper function to search road
144     BSTNode* searchHelper(BSTNode* node, string name) {
145         if (node == NULL || node->roadName == name) {
146             return node;
147         }
148 
149         if (name < node->roadName) {
150             return searchHelper(node->left, name);
151         } else {
152             return searchHelper(node->right, name);
153         }
154     }
```

```
156  public:
157      // Constructor
158      RoadSearchBST() {
159          root = NULL;}
160      // Insert road into BST
161      void insertRoad(string name, int dist) {
162          root = insertHelper(root, name, dist);
163          cout << "Road added to search system!" << endl;
164      }
165      // Search for a road
166      void searchRoad(string name) {
167          BSTNode* result = searchHelper(root, name);
168          if (result == NULL) {
169              cout << "Road not found!" << endl;
170          } else {
171      cout << "Road Found! Distance: "
172      << result->distance << " km" << endl;}}
173      // Display all roads in sorted order
174      void displayInorder() {
175      if (root == NULL) {
176      cout << "No roads in search system!" << endl;
177      return;}
178          cout << "\n===== ALL ROADS (SORTED) =====" << endl;
179          inorderHelper(root);}};
```

GRAPH CLASS (MAIN SYSTEM)

```
183 class Graph {
184 private:
185     int V; // Number of intersections
186     vector<Intersection> intersections;
187     vector<vector<int>> adj; // Adjacency list
188     vector<vector<int>> mat; // Adjacency matrix for distances
189     vector<vector<int>> trafficLevel; // Traffic congestion levels
190 public:
191     // Constructor
192     Graph(int vertices) {
193         V = vertices;
194         adj.resize(V);
195         mat.resize(V, vector<int>(V, 0));
196         trafficLevel.resize(V, vector<int>(V, 0));
197     }
198     // Add intersection to the city
199     void addIntersection(int id, string name) {
200         if (id >= V) {
201             cout << "Error: ID exceeds maximum vertices!" << endl;
202             return;
203         }
204         intersections.push_back(Intersection(id, name));
205         cout << "Intersection added: " << name << endl;
206     }
```

```
207     // Add road between two intersections (using your style)
208     void addEdge(int u, int v, int distance, int traffic) {
209         if (u >= V || v >= V) {
210             cout << "Error: Invalid intersection IDs!" << endl;
211             return;
212         }
213
214         adj[u].push_back(v);
215         adj[v].push_back(u); // Undirected graph
216
217         mat[u][v] = distance;
218         mat[v][u] = distance;
219
220         trafficLevel[u][v] = traffic;
221         trafficLevel[v][u] = traffic;
222
223         cout << "Road added between " << u << " and " << v << endl;
224     }
225
226     // Display city map (adjacency matrix)
227     void displayGraph() {
228         cout << "\n===== CITY MAP (ADJACENCY MATRIX) =====" << endl
229         cout << "   ";
230         for (int i = 0; i < V; i++) {
231             cout << i << " ";
232         }
```

```
224  // Display city map (adjacency matrix)
225  void displayGraph() {
226  cout << "\n===== CITY MAP (ADJACENCY MATRIX) =====" << endl;
227  cout << "   ";
228          for (int i = 0; i < V; i++) {
229              cout << i << " ";}
230          cout << endl;
231          for (int i = 0; i < V; i++) {
232              cout << i << ": ";
233              for (int j = 0; j < V; j++) {
234                  cout << mat[i][j] << " "; }
235              cout << endl;} }
236      // BFS traversal (using your style)
237  void bfsConnected(int src, vector<bool>& visited, vector<int>& res) {
238      queue<int> q;
239      visited[src] = true;
240      q.push(src);
241      while (!q.empty()) {
242      int curr = q.front();
243      q.pop();
244      res.push_back(curr);
245      // Visit all unvisited neighbors
246      for (int x : adj[curr]) {
247      if (!visited[x]) {
248                          visited[x] = true;
249                          q.push(x);
```

```
249 └ q.push(x);}}}}
250       // BFS for entire city
251       void bfs(int start) {
252           vector<bool> visited(V, false);
253           vector<int> res;
254           bfsConnected(start, visited, res);
255           cout << "\nBFS Traversal from " << start << ": ";
256           for (int i : res) {
257               cout << i << " ";}
258           cout << endl;}
259       // DFS traversal (using your style)
260       void dfsRec(vector<bool>& visited, int s, vector<int>& res) {
261           visited[s] = true;
262           res.push_back(s);
263           // Recursively visit all adjacent vertices
264           for (int i : adj[s]) {
265               if (visited[i] == false) {
266                   dfsRec(visited, i, res);}}}
267       // DFS for entire city
268       void dfs(int start) {
269           vector<bool> visited(V, false);
270           vector<int> res;
271           dfsRec(visited, start, res);
272           cout << "\nDFS Traversal from " << start << ": ";
273           for (int it : res) {
274               cout << it << " ";}
```

Dijkstra's algorithm for shortest path

```
278    void dijkstra(int src, int dest) {
279        vector<int> dist(V, INT_MAX);
280        vector<bool> visited(V, false);
281        vector<int> parent(V, -1);
282        dist[src] = 0;
283        for (int count = 0; count < V - 1; count++) {
284            // Find minimum distance vertex
285            int minDist = INT_MAX;
286            int u = -1;
287            for (int i = 0; i < V; i++) {
288                if (!visited[i] && dist[i] < minDist) {
289                    minDist = dist[i];
290                    u = i;
291             }}
292            if (u == -1) break;
293            visited[u] = true;
294            // Update distances of neighbors
295            for (int v = 0; v < V; v++) {
296                if (!visited[v] && mat[u][v] != 0 &&
297                    dist[u] != INT_MAX &&
298                    dist[u] + mat[u][v] < dist[v]) {
299                    dist[v] = dist[u] + mat[u][v];
300                    parent[v] = u;
301            }}}
```

```
304         // Display shortest path
305         if (dist[dest] == INT_MAX) {
306             cout << "No path exists!" << endl;
307             return; }
308         cout << "\nShortest distance from " << src << " to " << dest
309              << " = " << dist[dest] << " km" << endl;
310         cout << "Path: ";
311         vector<int> path;
312         int curr = dest;
313         while (curr != -1) {
314             path.push_back(curr);
315             curr = parent[curr];}
316         for (int i = path.size() - 1; i >= 0; i--) {
317             cout << path[i];
318             if (i != 0) cout << " -> ";}
319         cout << endl;}
320     // Detect congested roads
321     void detectCongestion() {
322         cout << "\n===== CONGESTED ROADS =====" << endl;
323         bool found = false;
324         for (int i = 0; i < V; i++) {
325     for (int j = i + 1; j < V; j++) {
326             if (trafficLevel[i][j] > 7) { // Threshold = 7
327           cout << "Road " << i << " <-> " << j
328         << " | Traffic Level: " << trafficLevel[i][j]
329         << " (CONGESTED!)" << endl;
```

```
331                     found = true;}  }}
332         if (!found) {
333             cout << "No congested roads found!" << endl;}}
334
335     // Display all intersections
336     void displayIntersections() {
337         if (intersections.empty()) {
338             cout << "No intersections added yet!" << endl;
339             return;
340         }
341
342         cout << "\n===== ALL INTERSECTIONS =====" << endl;
343         for (auto& it : intersections) {
344             it.displayInfo();
345         }
346     }
347
348     // Update traffic level on a road
349     void updateTraffic(int u, int v, int newLevel) {
350         if (u >= V || v >= V) {
351             cout << "Invalid intersection IDs!" << endl;
352             return;
353         }
```

```
        if (mat[u][v] == 0) {
            cout << "No road exists between these intersections!" << endl;
            return;
        }

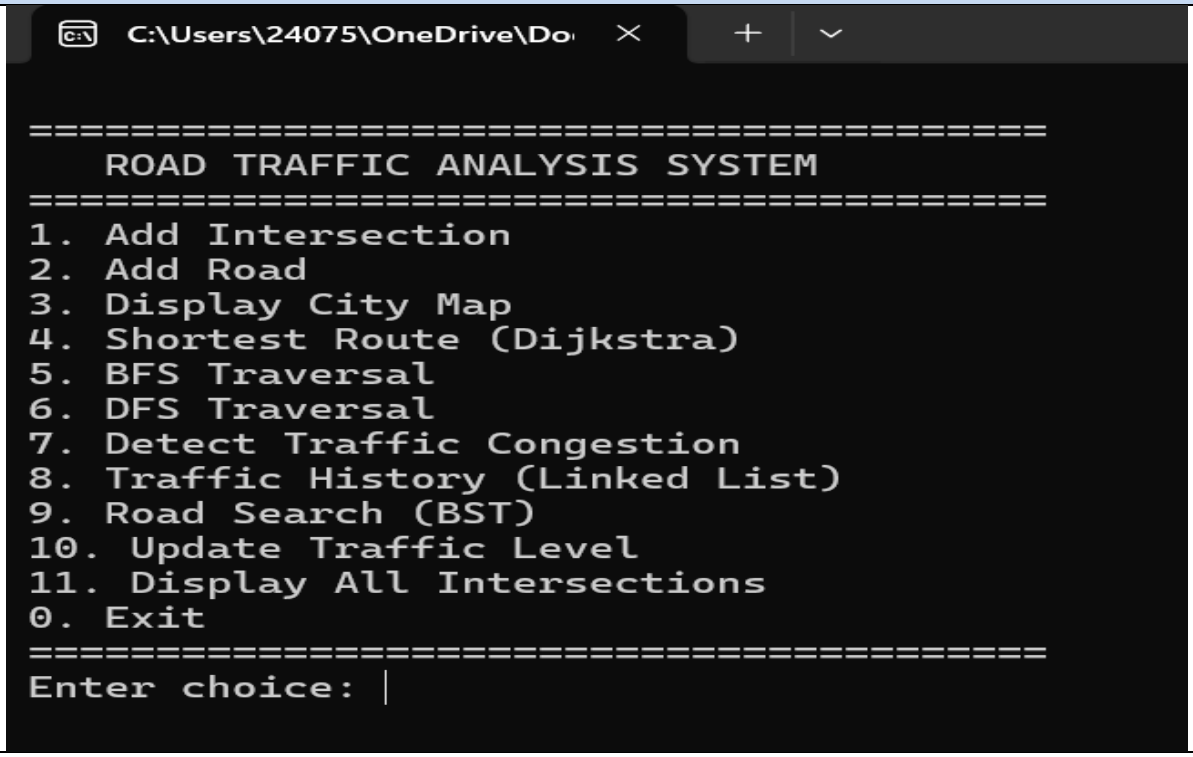
        trafficLevel[u][v] = newLevel;
        trafficLevel[v][u] = newLevel;
        cout << "Traffic level updated!" << endl;
    }
};
```

MAIN MENU(GUI)

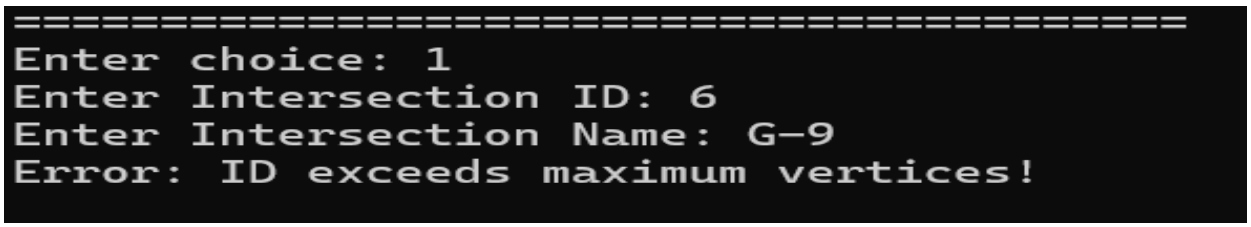
```
383    void showMainMenu() {
384        cout << "\n";
385        cout << "========================================" << endl;
386        cout << "   ROAD TRAFFIC ANALYSIS SYSTEM" << endl;
387        cout << "========================================" << endl;
388        cout << "1. Add Intersection" << endl;
389        cout << "2. Add Road" << endl;
390        cout << "3. Display City Map" << endl;
391        cout << "4. Shortest Route (Dijkstra)" << endl;
392        cout << "5. BFS Traversal" << endl;
393        cout << "6. DFS Traversal" << endl;
394        cout << "7. Detect Traffic Congestion" << endl;
395        cout << "8. Traffic History (Linked List)" << endl;
396        cout << "9. Road Search (BST)" << endl;
397        cout << "10. Update Traffic Level" << endl;
398        cout << "11. Display All Intersections" << endl;
399        cout << "0. Exit" << endl;
400        cout << "========================================" << endl;
401        cout << "Enter choice: ";
402    }
```

12. OUTPUT

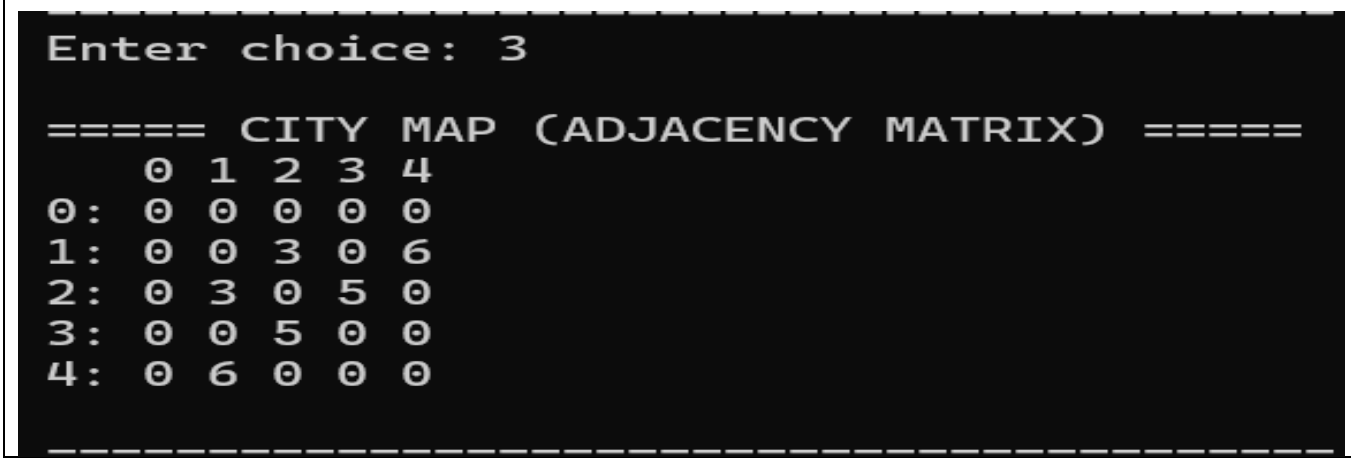
OUTPUT



ADD INTERSECTION



CTY MAP



❖ 13. Conclusion

❖ This project helped in understanding how data structures are applied to real-life problems like traffic management.

❖ The use of graph data structure made it easy to represent intersections and roads clearly.

❖ Linked lists were useful for storing and managing traffic history records dynamically.

❖ Binary Search Tree improved road searching compared to simple linear searching.

❖ Algorithms like BFS and DFS helped in traversing and analyzing the road network efficiently.

❖ Dijkstra's algorithm provided an effective way to find the shortest path between intersections.

❖ The menu-driven interface made the system easy to use and understand.

❖ The project improved problem-solving and logical thinking skills.

❖ This system also enhanced practical knowledge of C++ and object-oriented programming.

❖ Overall, the project successfully demonstrated the importance of data structures and algorithms in real-world applications.